



Self-Supervised Representation Learning & Contrastive Learning

Presenter: Mengyuan Wang
Date: 2026/09/03

Table of Contents

01 Research Background & Problem Formulation

02 Self-Supervised Learning

03 Contrastive Learning

04 Self-Distillation with NO labels: DINO

05 CLIP and Image-Text Alignment

01

Research Background & Problem Formulation

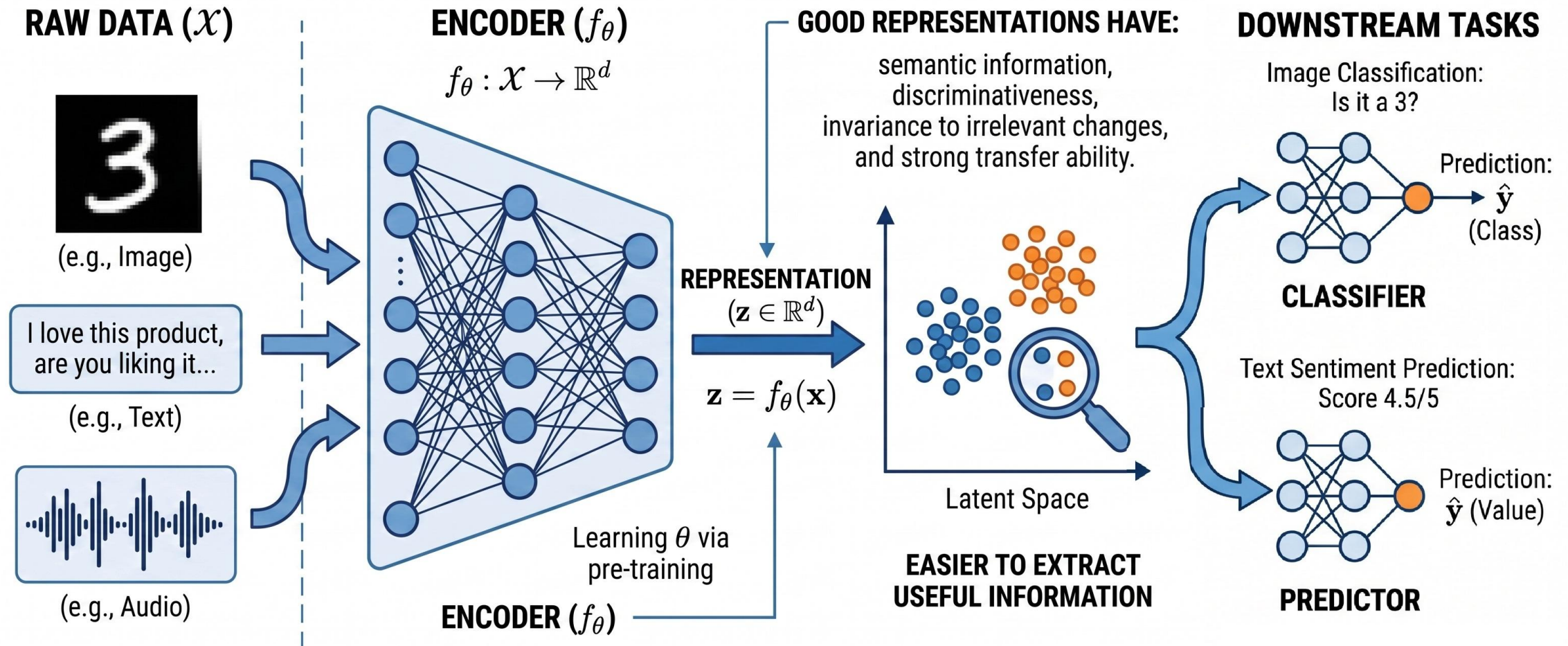
What is Representation Learning?



AVC Lab

Affective Vision Computing Lab

Representation Learning: *learning representations of the data that make it easier to extract useful information when building classifiers or other predictors.*



From Supervised Learning to Self-Supervised Learning



AVC Lab

Affective Vision Computing Lab



Supervised Learning

- Data: (x, y)
 x is data, y is label
- Goal: Learn a function to map $x \rightarrow y$
- Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

- Data: x
Just data, no labels!
- Goal: Learn hidden structure in data
- Examples: Clustering, dimensionality reduction, density estimation, etc.



Self-Supervised Learning

- Data: x
Raw data only, no human-annotated labels!
- Goal: Learn general representation via pretext tasks built from raw data
- Examples: Contrastive learning, reconstruction-based learning, predictive pretext tasks, etc.

Basic Goal of Self-Supervised Representation Learning

1. Automatically construct pseudo-supervisory signals

Generate pretext pre-training objectives from raw data, without human-annotated labels.

2. Learn semantically informative feature representations

Capture high-level semantic information of images, including object categories, spatial relationships and scene structures, rather than only low-level texture details.

3. Improve robustness and generalization

Learned representations remain stable against data augmentation, noise, color shifts and local appearance variations.

4. Task-agnostic property

Pre-training does not assume concrete downstream tasks; learned representations possess broad general applicability.

5. Enable transfer to downstream tasks

The pre-trained encoder can be transferred to classification, object detection, semantic segmentation, instance segmentation, object tracking, pose estimation and other tasks, especially for scenarios with limited annotated samples.

6. Evaluate representation quality

Validate the effectiveness of pre-trained representations via linear probing and full fine-tuning on downstream tasks.

02

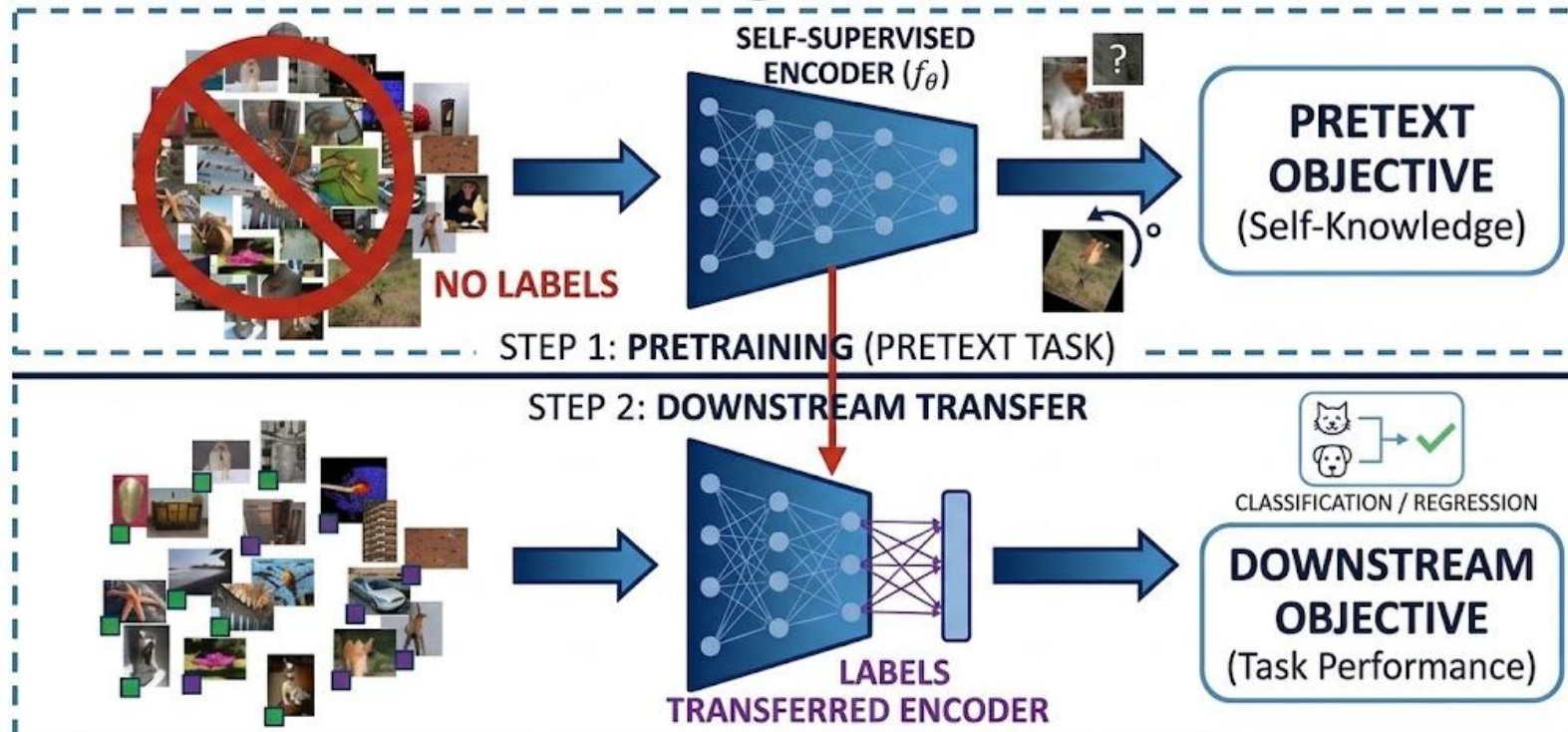
Self-Supervised Learning

Definition and Basic Pipeline of Self-Supervised Learning



AVC Lab

Affective Vision Computing Lab



03 Transfer to Downstream Tasks

- Freeze the pre-trained encoder as a feature extractor
- Fine-tune only the top-level classifier for downstream tasks (classification / detection)
- Greatly reduces annotation costs.
- The dataset is labeled



01 Construct Pretext Task

- Define a task based on the data it self
- No manual annotation
- Could be considered an unsupervised task
- But we learn with supervised learning objectives, e.g., classification or regression.



02 Pre-train the Encoder

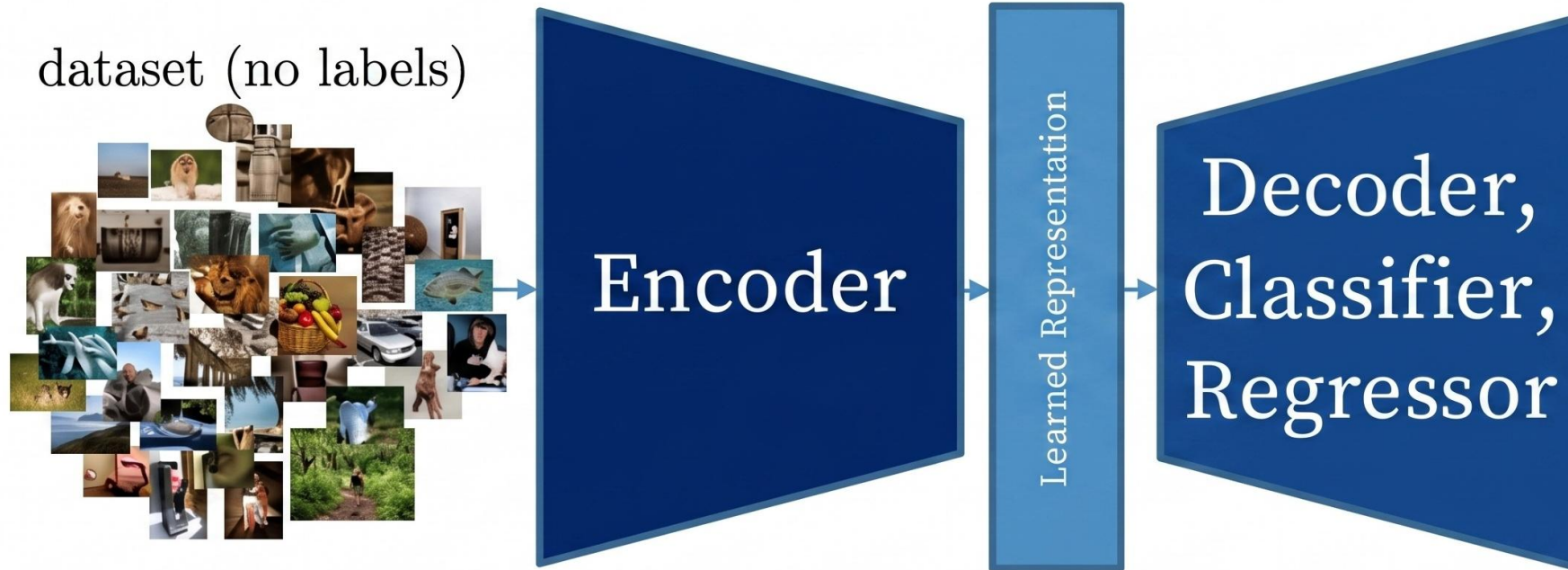
- Train the encoder network on massive unlabeled data
- Enable it to extract general image features
- Build a solid feature foundation for subsequent tasks.

Self-Supervised Learning – Pretext Task



AVC Lab

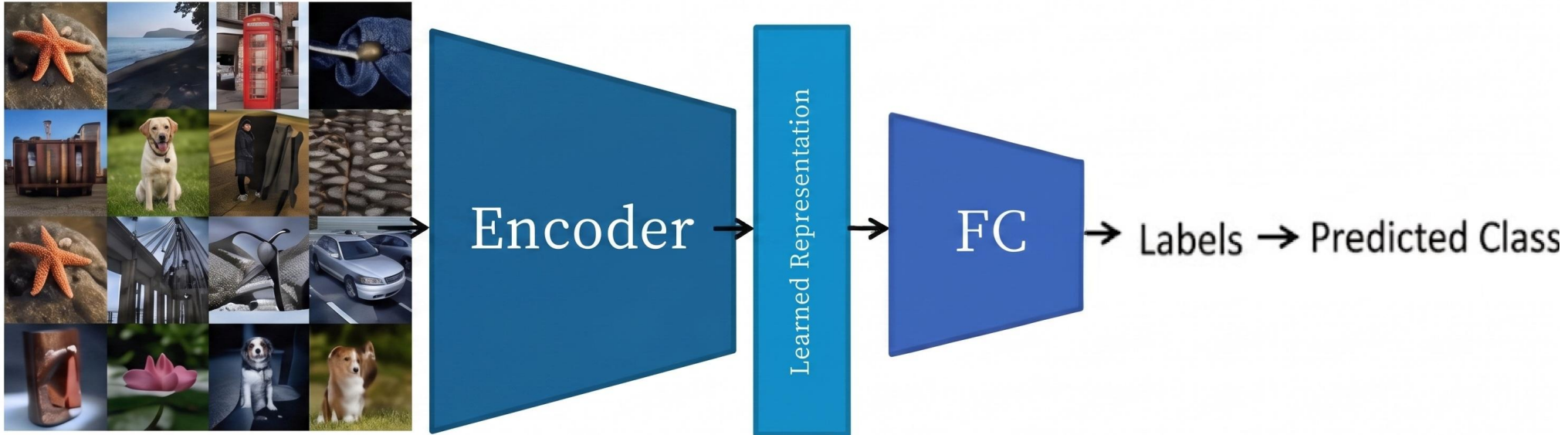
Affective Vision Computing Lab



labels/outputs automatically generated from data

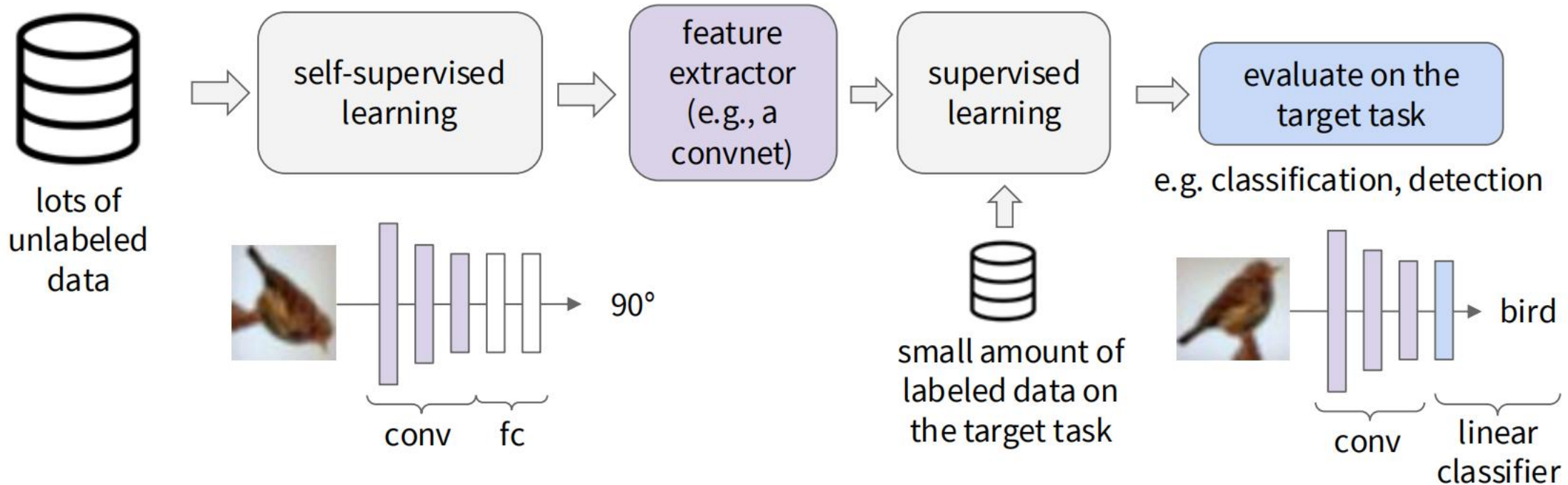
Self-Supervised Learning – Downstream Task

dataset (with labels)



Retain the pretrained encoder as a feature extractor and fine-tune it on downstream tasks.

Transfer Learning and Downstream Task Performance



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations.

<https://cs231n.stanford.edu/schedule.html>

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

Pretext Tasks from Image Transformations

- Rotation, Inpainting, Rearrangement, Coloring
- Reconstruction-based learning (MAE)

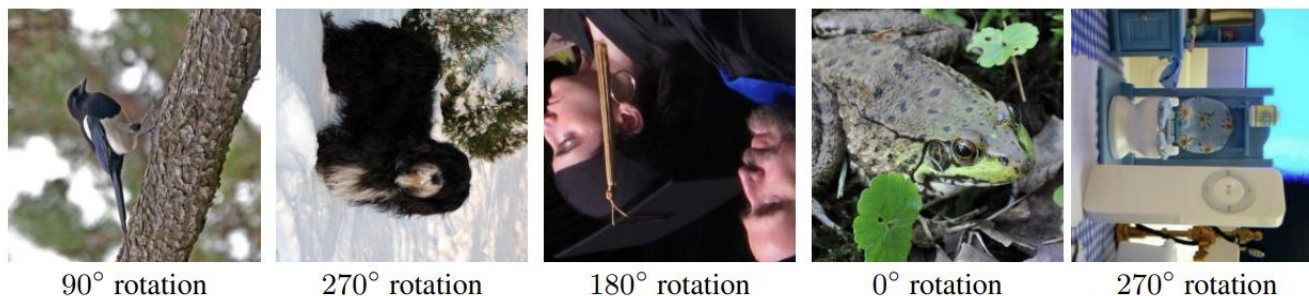
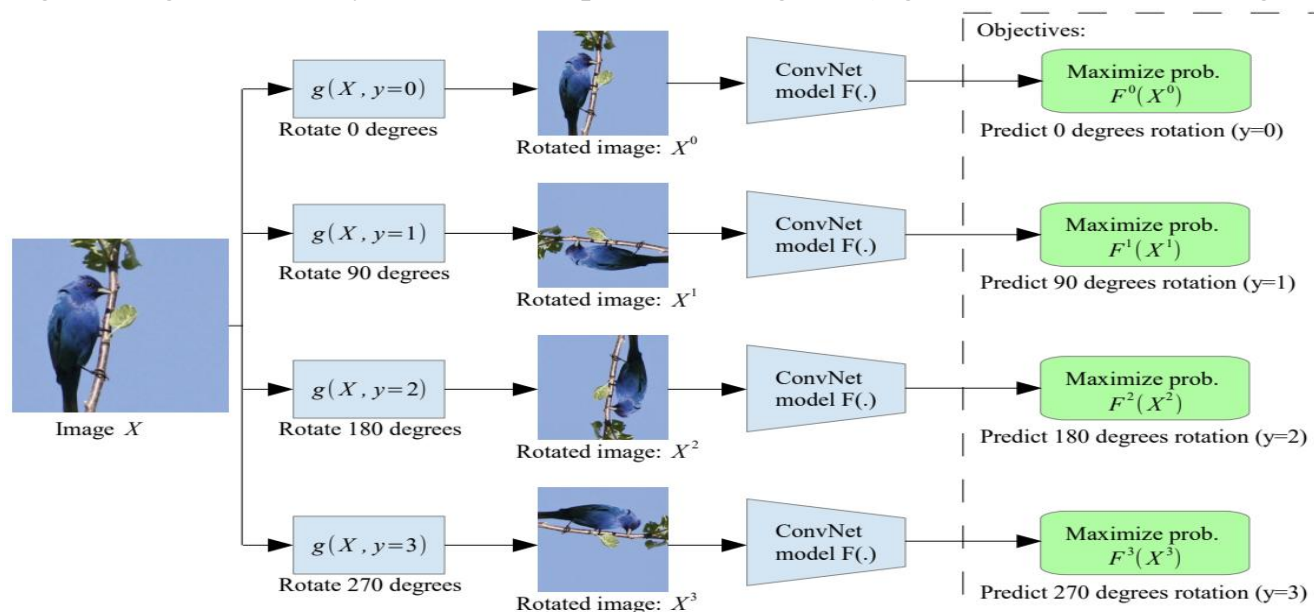


Fig 1: Images rotated by random multiples of 90 degrees (e.g., 0, 90, 180, or 270 degrees).



Rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification).

Pretext Tasks from Image Transformations



AVC Lab

Affective Vision Computing Lab

Inpainting



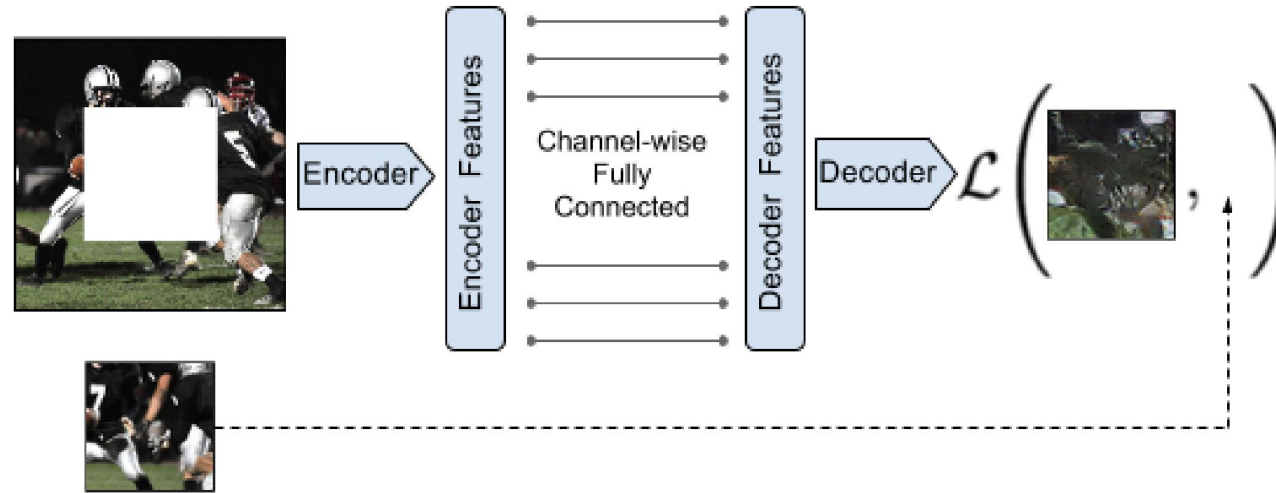
(a) Input context

(b) Human artist



(c) Context Encoder
(L2 loss)

(d) Context Encoder
(L2 + Adversarial loss)



- **Encoder (Feature Extraction):** An AlexNet-based backbone that encodes the surrounding context into a dense $6 \times 6 \times 256$ feature map.
- **Channel-wise Fully-Connected (Spatial Reasoning):** Propagates spatial information within each channel independently, enabling global reasoning while slashing parameter count by $256 \times$ compared to traditional FC layers.
- **1×1 Convolution (Channel Fusion):** A lightweight layer that blends representation across different channels to facilitate cross-feature communication.
- **Decoder (Semantic Generation):** A series of five deconvolutional layers that progressively upsample and hallucinate the missing pixels.

$$\text{Loss: } \mathcal{L} = \lambda_{rec} \mathcal{L}_{rec} + \lambda_{adv} \mathcal{L}_{adv}$$

<https://arxiv.org/pdf/1604.07379>

Pretext Tasks from Image Transformations



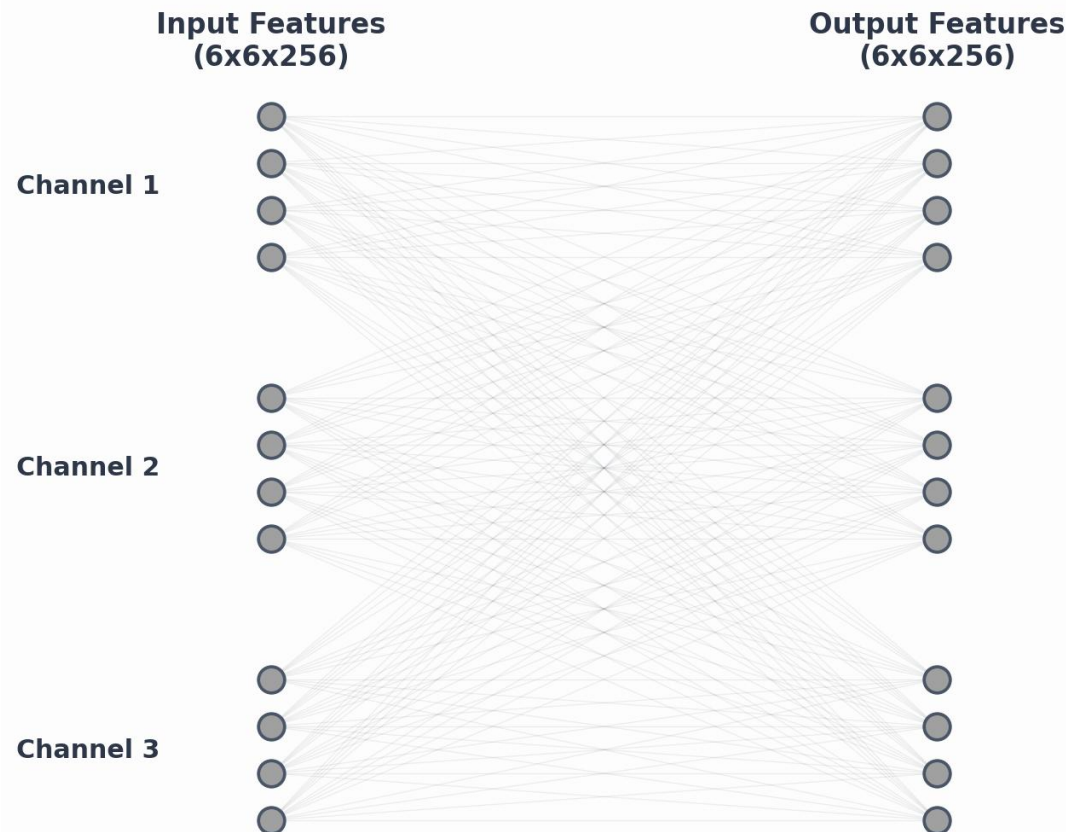
AVC Lab

Affective Vision Computing Lab

1. Traditional Fully-Connected Layer (Parameter Explosion)

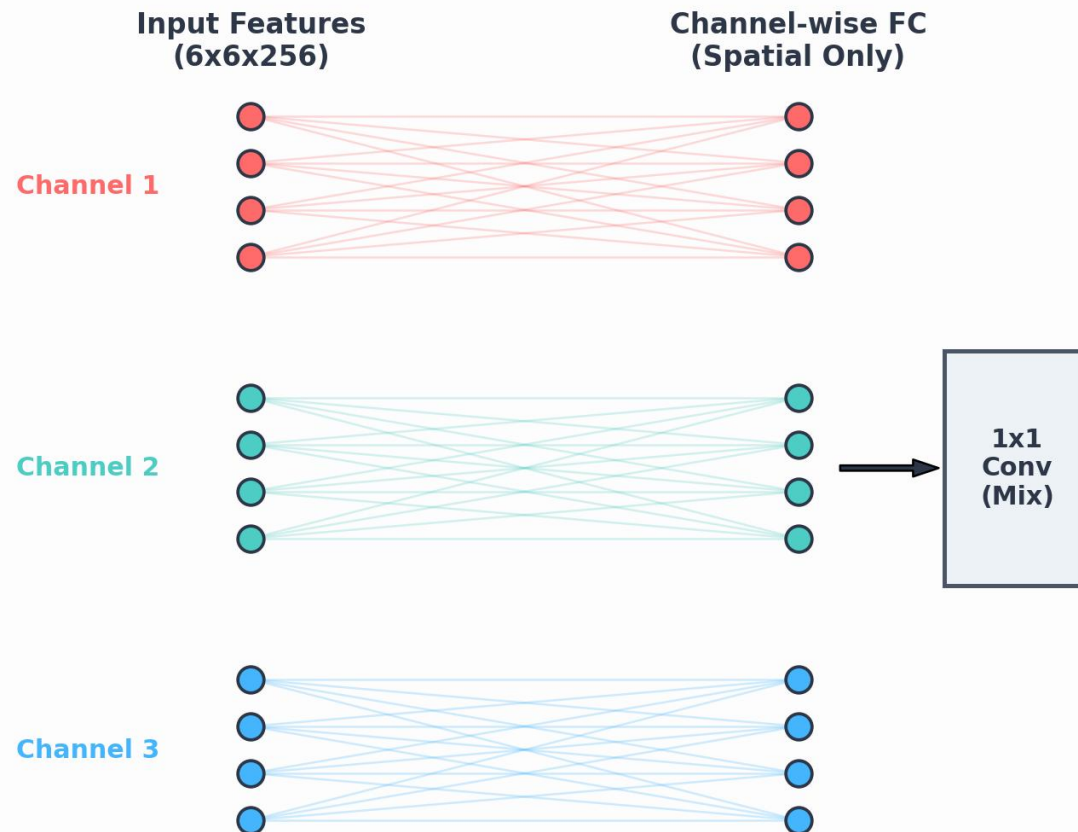
$$n \times n \times m$$

2. Channel-wise Fully-Connected Layer (parameters Slashed by 256×!)



Parameters: $m^2 \times n^4$

For $6 \times 6 \times 256$: $6 \times 6 \times 6 \times 6 \times 256 \times 256 = 84,934,656 \approx 85M$ parameters



Parameters: $m \times n^4 (+ 1 \times 1 \text{ Conv})$

For $6 \times 6 \times 256$: $6 \times 6 \times 6 \times 6 \times 256 + 256 \times 256 = 331,776 + 65536 \approx 331K$ parameters

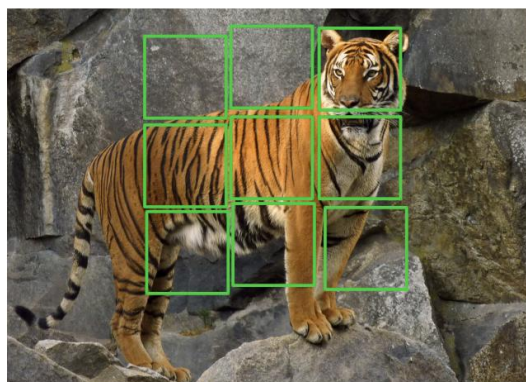
Pretext Tasks from Image Transformations

Rearrangement

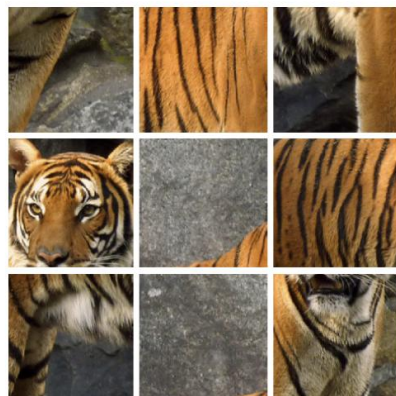


AVC Lab

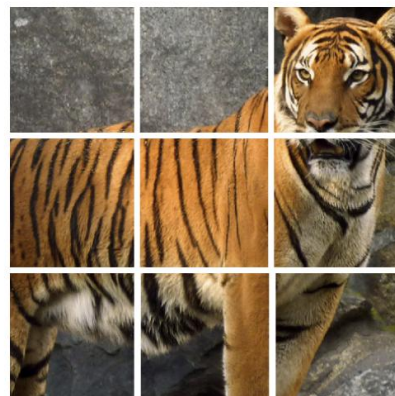
Affective Vision Computing Lab



(a)



(b)

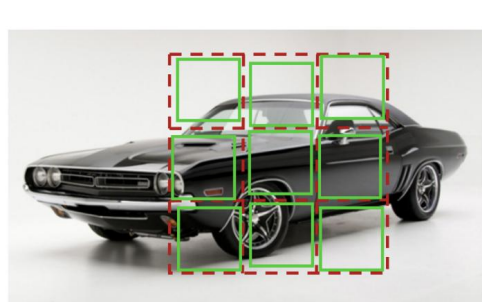


(c)

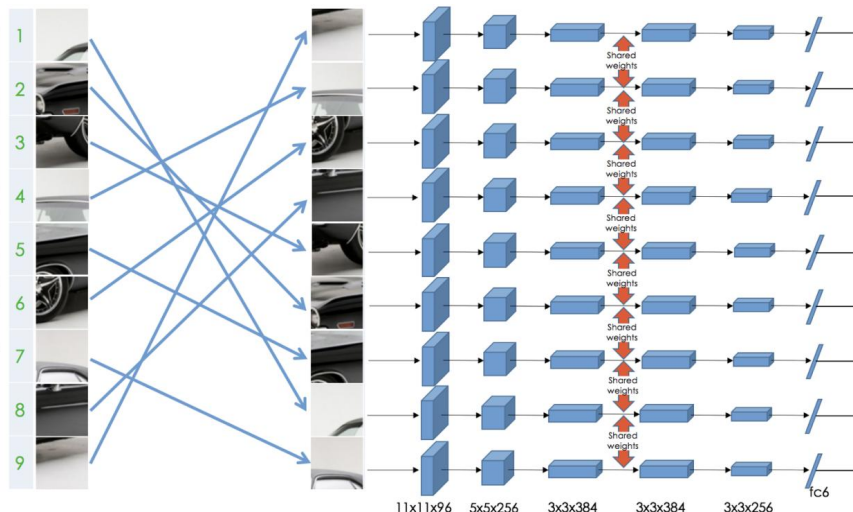
Architecture: Uses a Context-Free Network (CFN) consisting of 9 parallel, weight-sharing CNN branches that independently extract features from shuffled image tiles to prevent low-level shortcut learning.

Final Output: Outputs a probability distribution over a predefined set of spatial permutations (e.g., 100 or 1000 layouts) to classify and identify the correct jigsaw arrangement

Standard Cross-Entropy Loss



Permutation Set		Reorder patches according to the selected permutation
index	permutation	
64	9,4,6,8,3,2,5,1,7	



<https://arxiv.org/pdf/1603.09246>

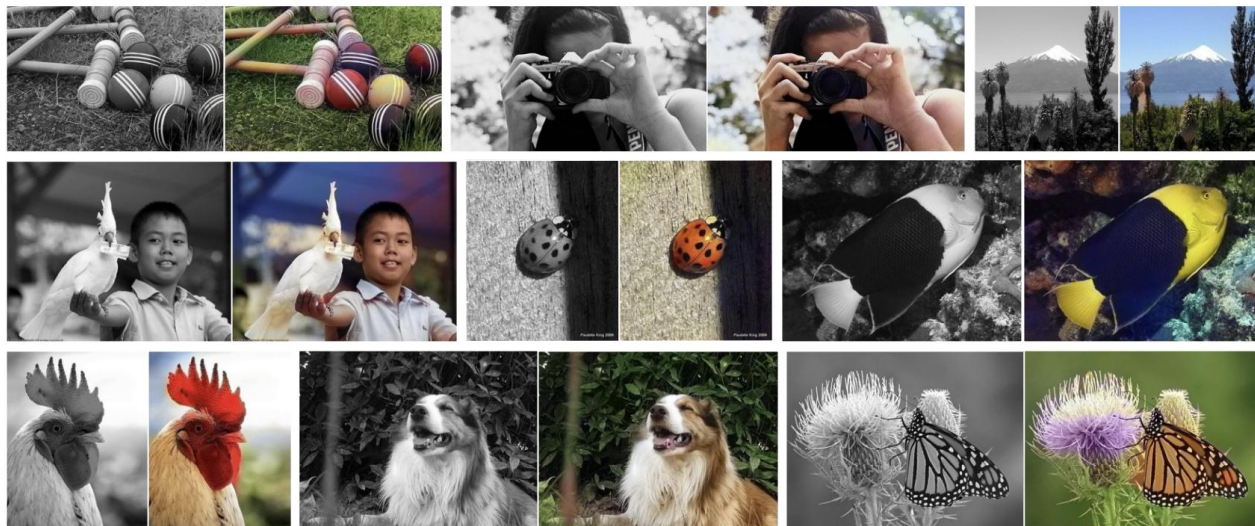
Pretext Tasks from Image Transformations

Coloring

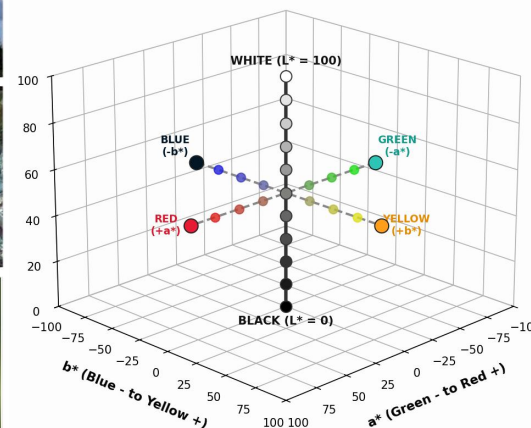


AVC Lab

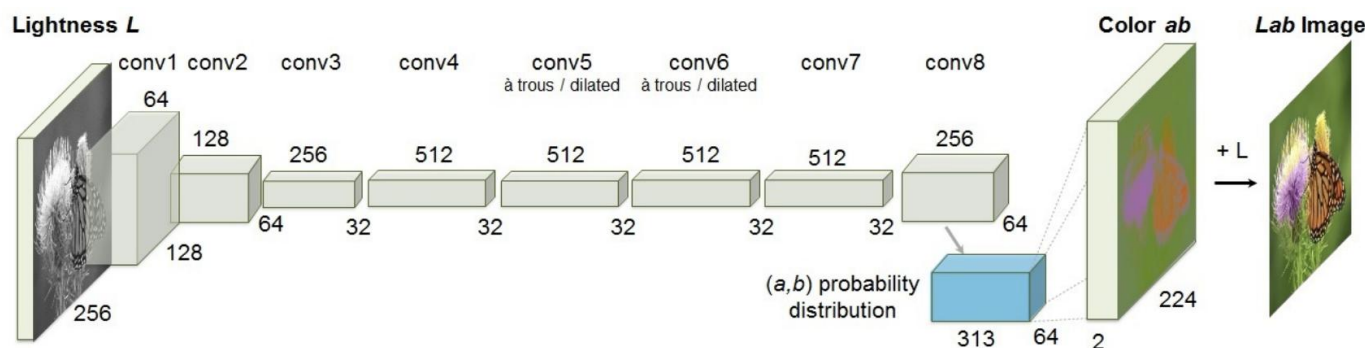
Affective Vision Computing Lab



CIE L*a*b* Color Space Structure



Color Space Transformation: Operates in **CIE Lab space** by **utilizing the single-channel Lightness L** as input to predict a probability distribution over quantized **ab** color channels.



Multinomial Classification: Discretizes the **ab** color space into **$Q = 313$** in-gamut bins, reformulating colorization from a regressive L2 optimization into a **pixel-wise multinomial classification task to capture color multimodality**.

¹Continuous $ab \in [-110, 110]^2$ Quantization (Step = 10) DenseGrid ($23 \times 23 = 529$) In-Gamut Filter $Q = 313$ Classes

Dilated VGG Backbone: Adopts a single-stream **VGG-style FCN without pooling layers**, leveraging dilated convolutions to **exponentially expand the receptive field** and **preserve high-resolution spatial context**.

<https://arxiv.org/abs/1603.08511>

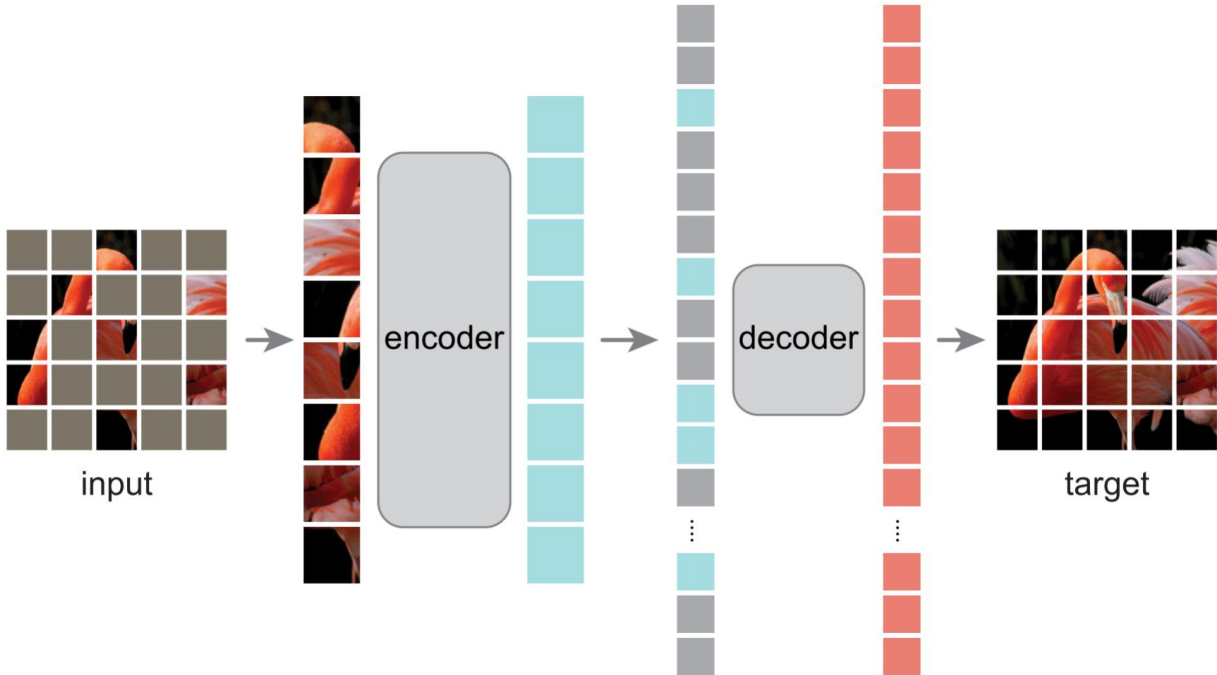
Reconstruction-based learning (MAE)

Masked Autoencoders



AVC Lab

Affective Vision Computing Lab



- **Masking:**

- Similar to the original ViT, divide the input into non-overlapping patches.
- Uniformly sample a **very large proportion** (75%) of these patches and mask them.
- **Masking a high ratio** makes predicting the task **challenging and meaning**.
- Not using mask tokens and picking a high sampling ratio (masking most of the image, e.g., 75%, and sampling a small visible portion, e.g., 25%, to feed into the encoder) allows to **train very large encoders** with **only a fraction of compute and memory**.

<https://arxiv.org/pdf/2111.06377>

- **Asymmetric Design:**

- Allows the encoder to operate only on the partial, observed signal (without mask tokens).
- A lightweight decoder that reconstructs the full signal from the latent representation and mask tokens.

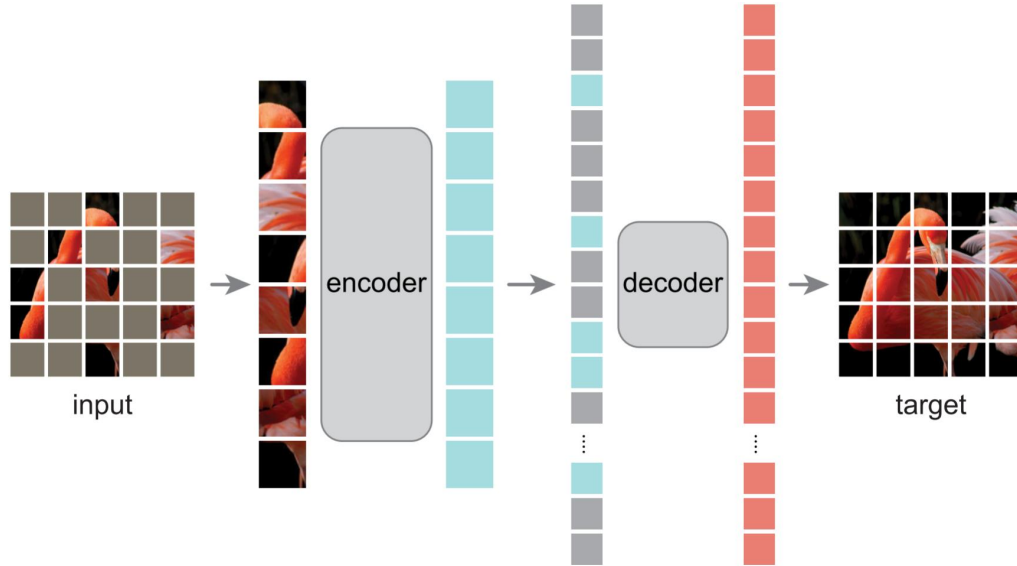
Reconstruction-based learning (MAE)

Masked Autoencoders



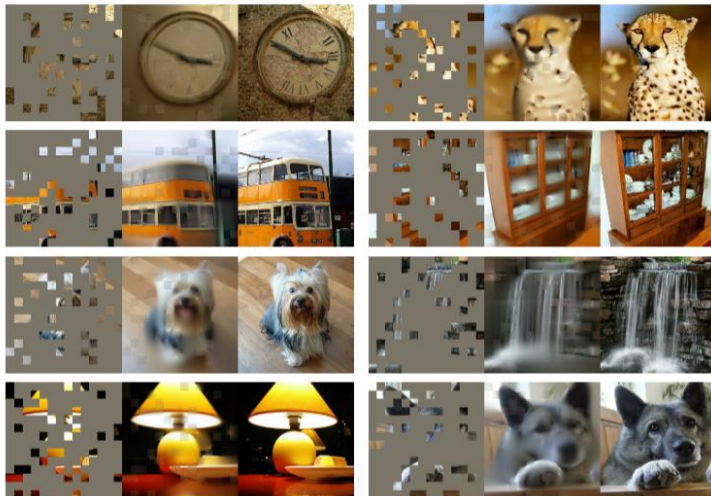
AVC Lab

Affective Vision Computing Lab



- **MAE Encoder:**

- The encoder only operates on unmasked patches (25%)
- Embeds patches by a linear projection with added positional embeddings.
- Uses transformer block
- Since the input patches are a small part of the input, the encoder is chosen to be **very large**.



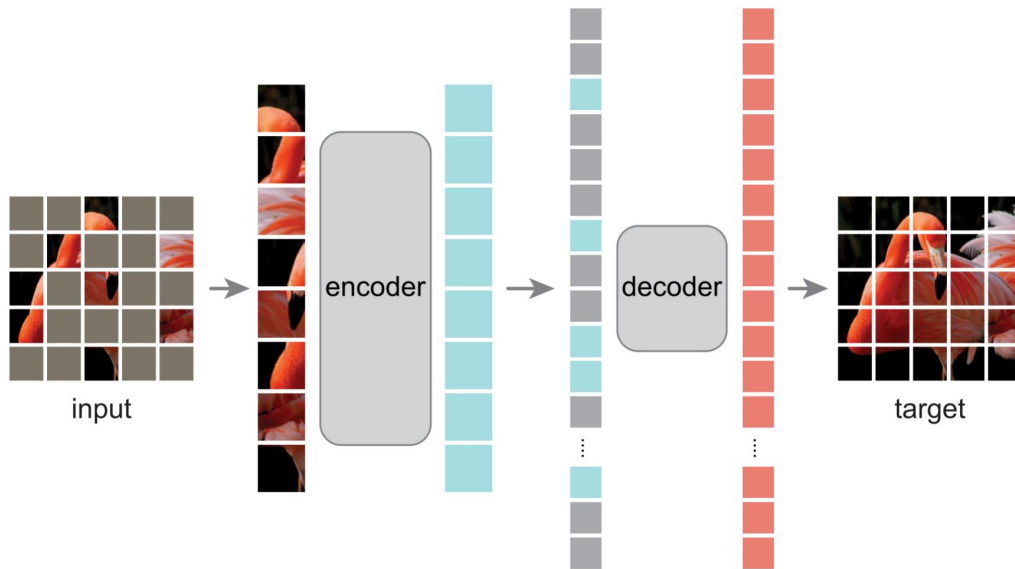
Reconstruction-based learning (MAE)

Masked Autoencoders



AVC Lab

Affective Vision Computing Lab



- **MAE Decoder (lightweight):**
 - Merges the encoder outputs with the **shared mask tokens** in previously masked places, adding positional encodings to them.
 - Uses transformer blocks, followed by a **linear projection for finalizing pixel reconstruction**. Number of output channels equals the number of pixel values in a patch.
 - **The MAE decoder is only used during pre-training** to perform the image reconstruction task (only the encoder is used to produce image representations for recognition).
 - **The full set of tokens are only processed by the lightweight decoder**, which significantly reduces pre-training time.

<https://arxiv.org/pdf/2111.06377>

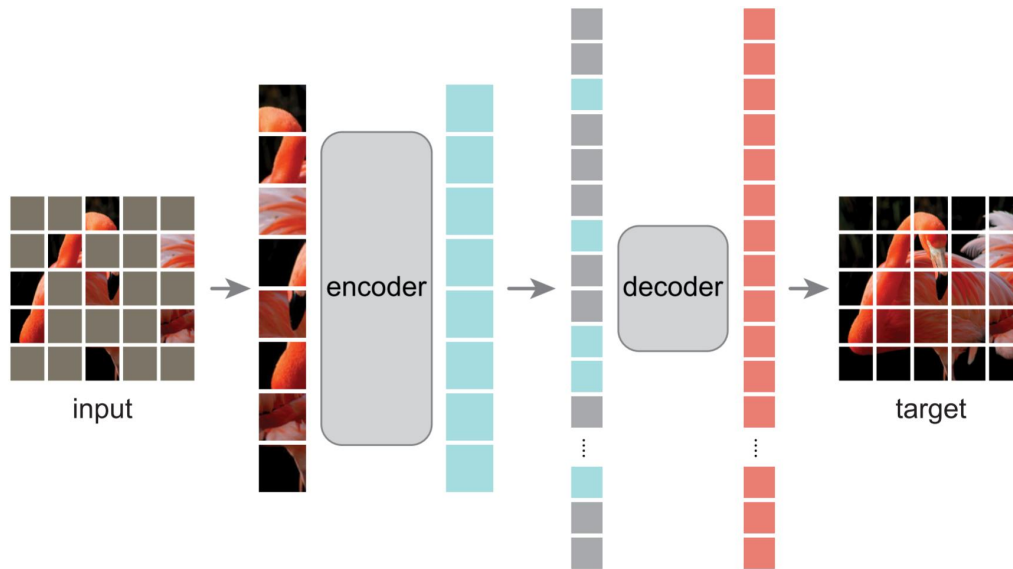
Reconstruction-based learning (MAE)

Masked Autoencoders



AVC Lab

Affective Vision Computing Lab



- **Reconstruction target:**
 - **Predict the pixel values** for each masked patch
 - The **MSE (mean squared error loss)** in the **pixel space** between the input image and the reconstructed image is adopted.
 - The loss is only computed for masked patches.

03

Contrastive Learning

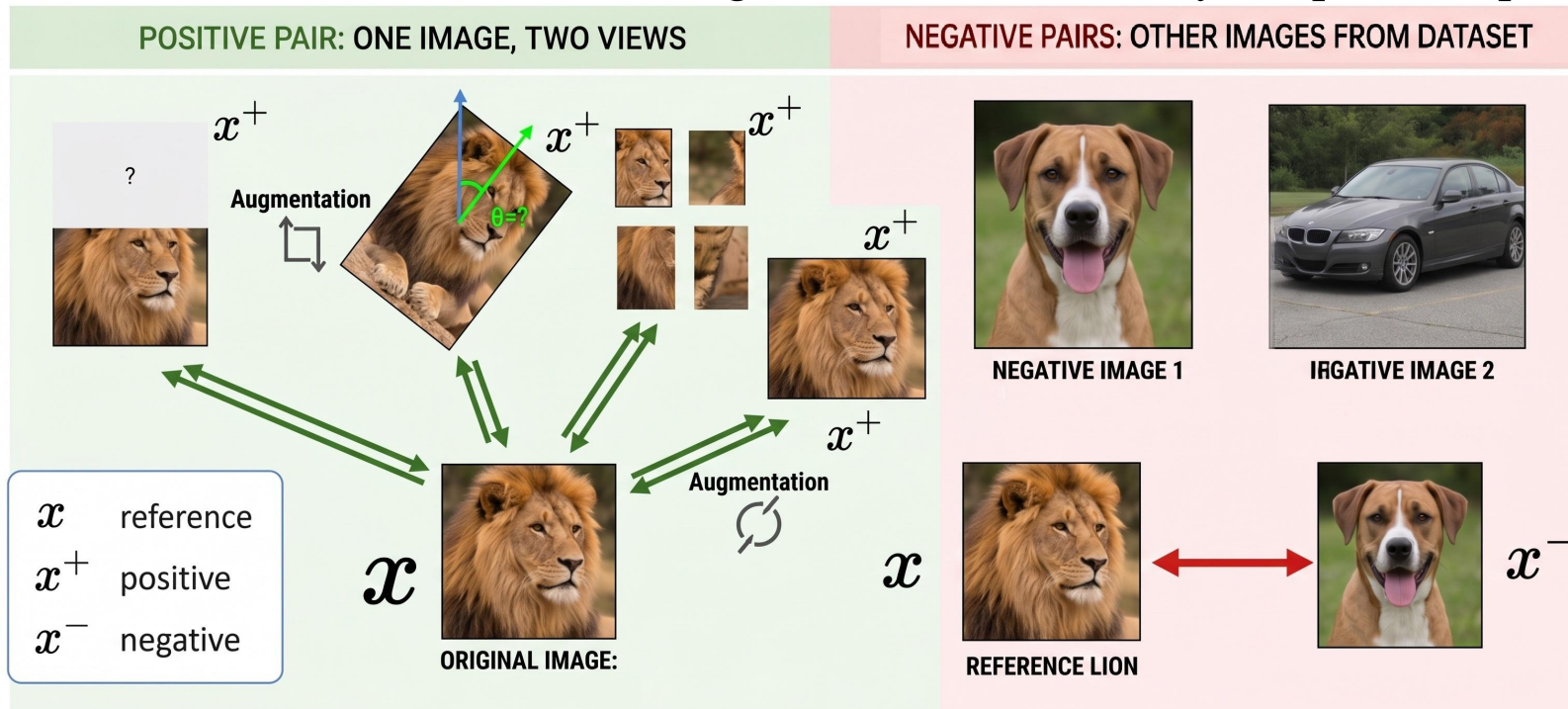
Fundamental Principles of Contrastive Learning



AVC Lab

Affective Vision Computing Lab

- **Objective:** Learn an embedding space $f_{\theta}(\mathbf{x})$
 - Similar samples (**Positive Pairs**) are placed close to each other.
 - Dissimilar samples (**Negative Pairs**) are far apart in the representation space.
- **Positive Pairs:** Two correlated views of the same image via random augmentations (random cropping, color jitter). They are pulled close in latent space.
- **Negative Pairs:** Views from different images in the dataset. They are pushed apart in latent space.



Loss Function for Contrastive Learning

Contrastive Loss



AVC Lab

Affective Vision Computing Lab

$$\mathcal{L}_{cont}(x_i, x_j, \theta) = \mathbb{1}[y_i = y_j] \|f_{\theta}(x_i) - f_{\theta}(x_j)\|_2^2 + \mathbb{1}[y_i \neq y_j] \max(0, \epsilon - \|f_{\theta}(x_i) - f_{\theta}(x_j)\|_2)^2$$

Positive Pair Scenario ($y_i = y_j$):

- The indicator functions evaluate to $\mathbb{1}[y_i = y_j] = 1$ and $\mathbb{1}[y_i \neq y_j] = 0$.
- The loss simplifies to $\mathcal{L} = D^2 = \|f_{\theta}(x_i) - f_{\theta}(x_j)\|_2^2$.
- Minimizing this loss **pulls identical-class representations together toward zero distance**.

Negative Pair Scenario ($y_i \neq y_j$):

- The indicator functions evaluate to $\mathbb{1}[y_i = y_j] = 0$ and $\mathbb{1}[y_i \neq y_j] = 1$.
- The loss simplifies to $\mathcal{L} = \max(0, \epsilon - D)^2$.
- If $D \geq \epsilon$, the loss is 0 (representations are sufficiently separated).
- If $D < \epsilon$, the loss is $(\epsilon - D)^2 > 0$, generating a gradient to push different-class representations apart until $D \geq \epsilon$.

Loss Function for Contrastive Learning

Triplet Loss



AVC Lab

Affective Vision Computing Lab

$$\mathcal{L}_{triplet}(x, x^+, x^-) = \sum_{x \in \mathcal{X}} \max(0, \|f(x) - f(x^+)\|_2^2 - \|f(x) - f(x^-)\|_2^2 + \epsilon)$$

Triplet: anchor: x positive: x^+ negative: x^-

Let the squared Euclidean distance from the anchor to the positive sample be $D_{ap}^2 = \|f(\mathbf{x}) - f(\mathbf{x}^+)\|_2^2$, and from the anchor to the negative sample be $D_{an}^2 = \|f(\mathbf{x}) - f(\mathbf{x}^-)\|_2^2$.

Optimization Objective & Margin Constraint

- The objective is to ensure the anchor-positive distance is smaller than the anchor-negative distance by at least a margin ϵ :

$$D_{ap}^2 + \epsilon \leq D_{an}^2 \Leftrightarrow D_{ap}^2 - D_{an}^2 + \epsilon \leq 0$$

- Formulating this via hinge loss yields $\mathcal{L} = \max(0, D_{ap}^2 - D_{an}^2 + \epsilon)$:
 - When $D_{an}^2 \geq D_{ap}^2 + \epsilon$, the margin condition is satisfied and $\mathcal{L} = 0$.
 - When $D_{an}^2 < D_{ap}^2 + \epsilon$, $\mathcal{L} > 0$, generating gradients that simultaneously decrease D_{ap} and increase D_{an} .

Loss Function for Contrastive Learning

N pair Loss



AVC Lab

Affective Vision Computing Lab

$$\mathcal{L}_{N\text{-pair}} = -\log \frac{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+))}{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+)) + \sum_{j=1}^{N-1} \exp(f(\mathbf{x})^\top f(\mathbf{x}_j^-))}$$

Tuple Input: Formulated with 1 anchor $\mathbf{x}$, 1 positive $\mathbf{x}^+$, and $N - 1$ negative samples $\{\mathbf{x}_j^-\}_{j=1}^{N-1}$.

Numerator: Exponential similarity score $\exp(s^+)$ of the positive pair.

Denominator: Total partition function summing over 1 positive and all $N - 1$ negative pairs.

Core Equivalence: Formulates deep metric learning as a dynamic **N-class Softmax Cross-Entropy objective**.

Loss Function for Contrastive Learning

Information Noise-Contrastive Estimation (InfoNCE)

Goal: Pull positive pairs closer, push negative pairs apart via multi-class classification.

$$\mathcal{L}_{InfoNCE} = -\log \frac{\exp(\text{sim}(q, k_+)/\tau)}{\exp(\text{sim}(q, k_+)/\tau) + \sum_{i=1}^K \exp(\text{sim}(q, k_i^-)/\tau)}$$

- **sim(u, v):** Cosine similarity $\frac{u^T v}{\|u\|_2 \|v\|_2}$
- **Equivalent to:** $(K + 1)$ -class Softmax Cross-Entropy

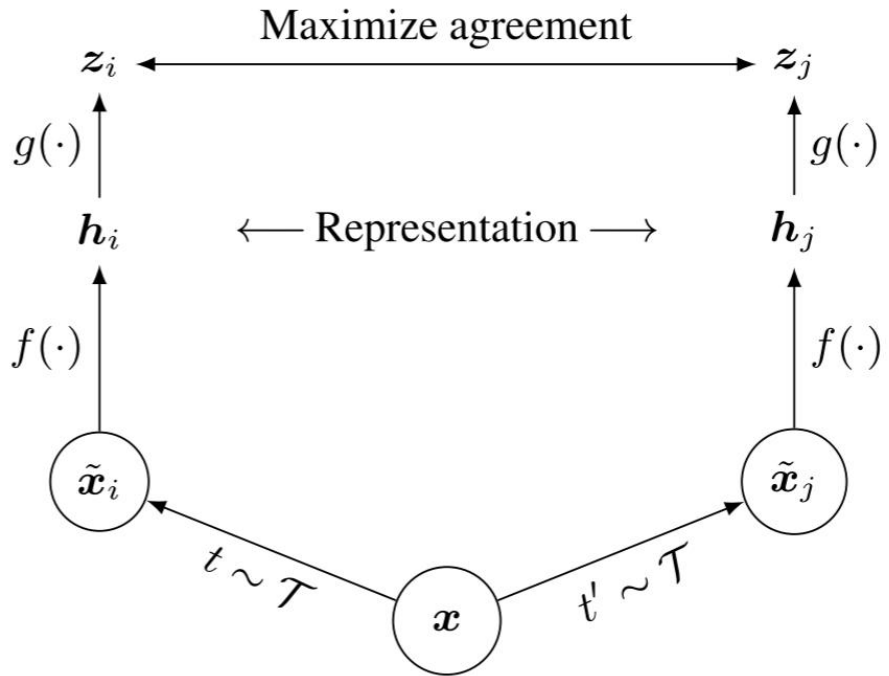
Key Properties:

- **Mutual Information Bound:** Maximizes $I(q; k_+) \geq \log(K) - \mathcal{L}_{InfoNCE}$ (more negatives $K \rightarrow$ tighter bound)
- **Temperature τ :** Scales logit sharpness; smaller τ penalizes hard negatives more heavily.

SimCLR: A Simple Framework for Contrastive Learning of Visual Representations



Affective Vision Computing Lab



SimCLR learns representations by **maximizing agreement between differently augmented views of the same data** example via a **contrastive loss** in the **latent space**.

Cosine similarity as the score function:

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}$$

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random Gaussian blur.

A neural network base encoder $f(\cdot)$ that extracts representation vectors from augmented data examples. $h_i = f(\tilde{x}_i) = \text{ResNet}(\tilde{x}_i)$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied. $z_i = g(h_i) = W^{(2)} \sigma(W^{(1)} h_i)$, where σ is a ReLU non-linearity.

Given a set $\{\tilde{x}_k\}$ including a positive pair of examples $\tilde{x}_i$ and $\tilde{x}_j$, the contrastive prediction task aims to identify $\tilde{x}_j$ in $\{\tilde{x}_k\}_{k \neq i}$ for a given $\tilde{x}_i$.

SimCLR: A Simple Framework for Contrastive Learning of Visual Representations

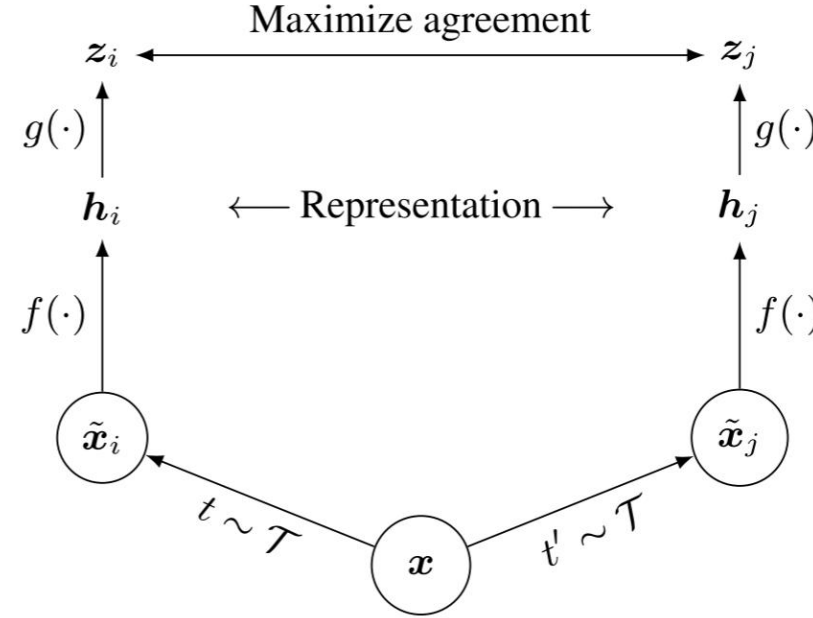


AVC Lab

Affective Vision Computing Lab

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$



Treat the other $2(N - 1)$ augmented examples within a minibatch as negative examples.

Then the loss function for a positive pair of examples (i, j) is defined as

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

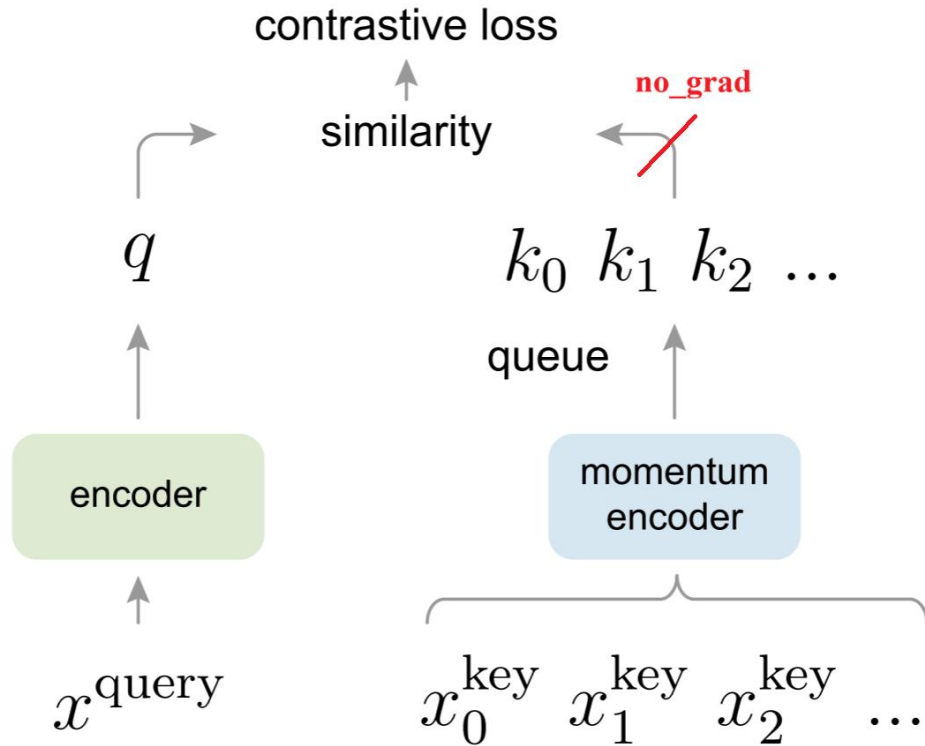
Contrastive learning benefits from **larger batch sizes** and **more training steps** compared to supervised learning.

MoCo: Momentum Contrast



AVC Lab

Affective Vision Computing Lab



Encoder: f_q, f_k (ResNet)

Data Augmentation Setting: random resized 224×224 crop, random color jitter, horizontal flip, random grayscale conversion.

Positive Pair: Take two random “views” of the same image under random data augmentation. $x^q, x^k \rightarrow f_q, f_k \rightarrow q = f_q(x^q), k_+ = f_k(x^k)$

Negative Keys: Maintained as a dynamic FIFO queue of K keys
(out-of-batch keys) $x^k \rightarrow f_k \rightarrow k = f_k(x^k) \rightarrow \{k_0, k_1, k_2, \dots\}$

InfoNCE Loss: $\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k_i / \tau)}$ $\tau = 0.07$

Key difference to SimCLR:

- Keep a running queue of keys (negative samples).
- Compute gradients and update the encoder only through the queries.
- Decouple min-batch size with the number of keys: can support a large number of negative samples.
- The key encoder is slowly progressing through the momentum update rules:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$

MoCo: Momentum Contrast



AVC Lab

Affective Vision Computing Lab

Generate a positive pair by sampling data augmentation functions

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature
```

```
f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version
```

```
q = f_q.forward(x_q) # queries: Nx C
k = f_k.forward(x_k) # keys: Nx C
k = k.detach() # no gradient to keys
```

```
# positive logits: Nx1
l_pos = bmm(q.view(N,1,C), k.view(N,C,1))
```

```
# negative logits: NxK
l_neg = mm(q.view(N,C), queue.view(C,K))
```

```
# logits: Nx(1+K)
logits = cat([l_pos, l_neg], dim=1)

# contrastive loss, Eqn. (1)
labels = zeros(N) # positives are the 0-th
loss = CrossEntropyLoss(logits/t, labels)
```

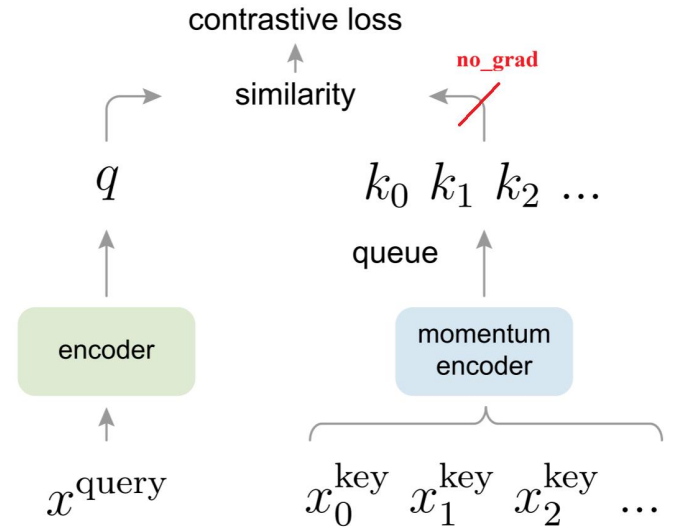
```
# SGD update: query network
loss.backward()
update(f_q.params)
```

```
# momentum update: key network
f_k.params = m*f_k.params+(1-m)*f_q.params
```

```
# update dictionary
enqueue(queue, k) # enqueue the current minibatch
dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

Use the running queue of keys as the negative samples



InfoNCE loss

Update f_k through momentum

No gradient through the key

Update the FIFO negative sample queue

04

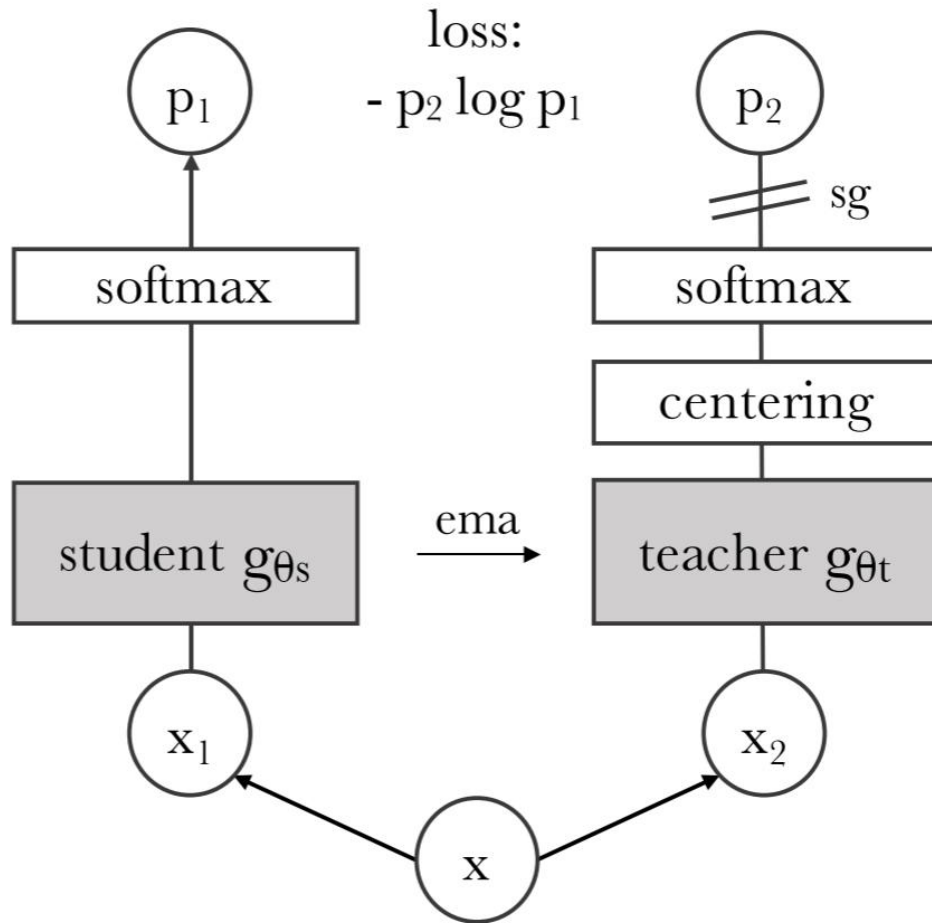
Self-Distillation with NO labels: DINO

Self-Distillation with **NO** labels: DINO



AVC Lab

Affective Vision Computing Lab



Asymmetric Student-Teacher Framework:

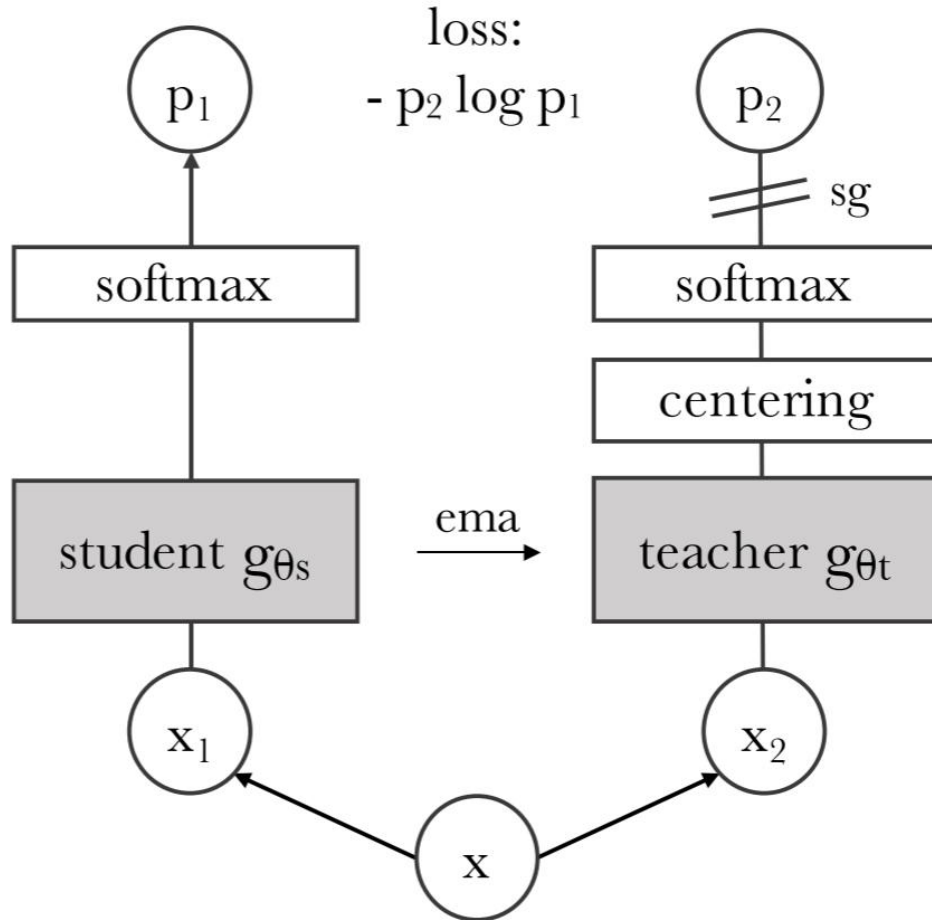
- **Structural Design:** Student network g_{θ_s} is optimized via backpropagation; teacher network g_{θ_t} applies a **stop-gradient** operator and receives no gradients directly.
- **Network Architecture:** The neural network g is composed of a **backbone f (ViT or ResNet)**, and of a **projection head h** : $g = h \circ f$.
- **Teacher Update:** Teacher weights θ_t are updated as an **Exponential Moving Average (EMA)** of the student parameters: $(\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s)$. **Due to ensembling effects, the teacher consistently outperforms the student during training.**
- **BN-Free Architecture:** No Batch Normalization (BN) is utilized in the projection heads, creating a highly efficient and stable fully BN-free pretraining pipeline.

Self-Distillation with **NO** labels: DINO



AVC Lab

Affective Vision Computing Lab



Local-to-Global Multi-Crop Correspondence:

- **Global Views:** 2 high-resolution crops (224×224) covering ($> 50\%$) of the original image, passed to **both Student and Teacher**.
- **Local Views:** Multiple lower-resolution crops (96×96) covering ($< 50\%$) of the image, passed strictly to the **Student**.
- **Training Objective:** Guided by standard cross-entropy loss, the Student is forced to predict global, macro-level scene representations using only local, micro-level views, establishing robust spatial-semantic correspondences.

Self-Distillation with **NO** labels: DINO



Affective Vision Computing Lab

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Avoiding Collapse: Centering & Sharpening Balances

- **Centering:** Subtracts the batch mean center c from the teacher's outputs. This prevents any single dimension from dominating (avoiding "one-hot dimension collapse") but encourages a drift towards a uniform distribution.
- **Sharpening:** Uses a low temperature value (τ_t) (ranging from 0.04 to 0.07) in the teacher's softmax normalization. This scales down entropy, making the distribution highly sharp and discriminative to prevent uniform collapse.
- **Dynamic Counterweight:** The interplay of centering and sharpening successfully balances optimization **without requiring negative pairs, advanced normalizations, or predictors.**

05

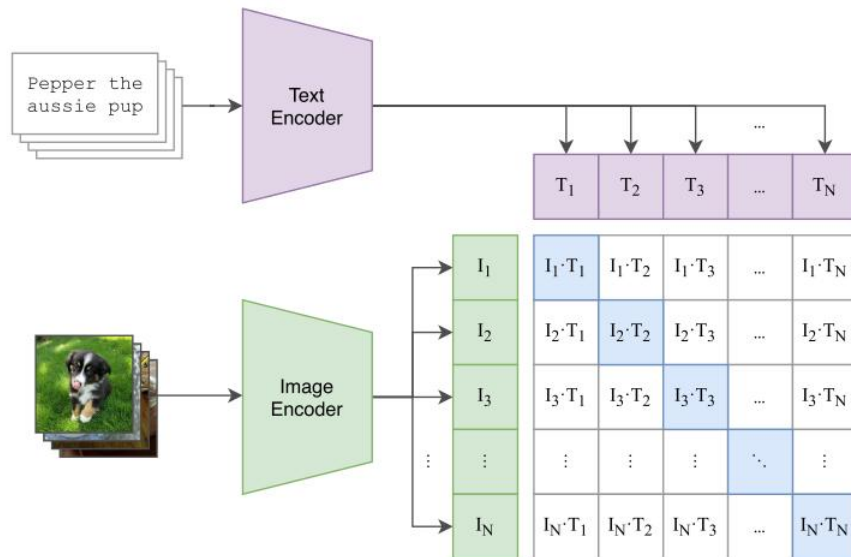
CLIP and Image-Text Alignment

CLIP: Contrastive Language-Image Pre-training



Affective Vision Computing Lab

(1) Contrastive pre-training



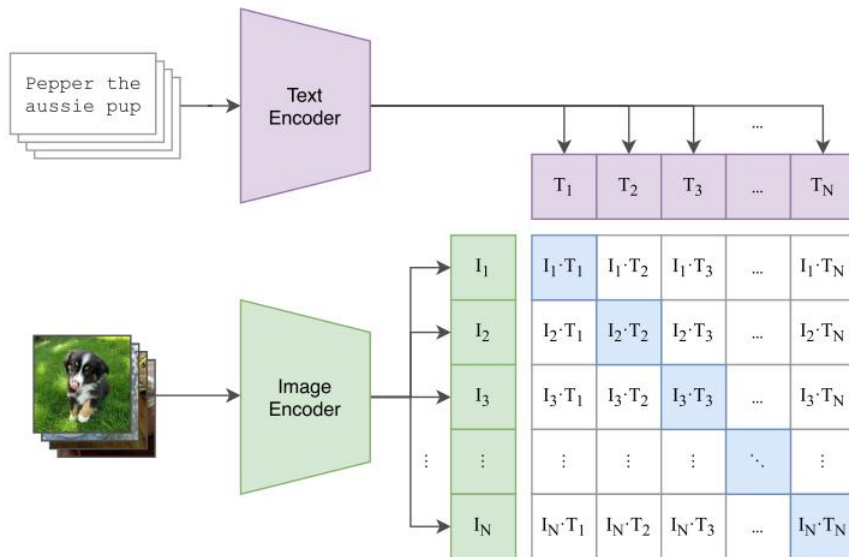
- **Pre-training on Massive Web-scale Data:** 400 million (image, text) pairs
- **Proxy Contrastive task:** Predict whether images within the current mini-batch are matched with full-sentence texts.
- **Image Encoder:** Supports classic ResNet variants (e.g., RN50, RN101, RN50x64) and Vision Transformers (e.g., ViT-B/32, ViT-L/14) to extract deep visual representations.
- **Text Encoder:** A standard Text Transformer designed to embed textual inputs into corresponding representations
- **A Linear Projection:** Map from each encoder's representation to the multi-modal embedding space.
- **Train CLIP from scratch:** Without initializing the image encoder with ImageNet weights or the text encoder with pre-trained weights.

CLIP: Contrastive Language-Image Pre-training



Affective Vision Computing Lab

(1) Contrastive pre-training



- **Symmetric Contrastive Loss:** Given a batch of N (image, text) pairs, CLIP maximizes the **cosine similarity** of the N true pairings while minimizing the $N^2 - N$ incorrect pairings, optimizing a **symmetric cross-entropy loss**.
- **Input:** A mini-batch of N ground-truth image-text pairs.
- **Image Features:** $I = \{I_1, I_2, \dots, I_N\}$, where $I_i \in \mathbb{R}^C$ and $\|I_i\|_2 = 1$ (L_2 -normalized).
- **Text Features:** $T = \{T_1, T_2, \dots, T_N\}$, where $T_j \in \mathbb{R}^C$ and $\|T_j\|_2 = 1$ (L_2 -normalized).
- **The cosine similarity between the i – th image and the j – th text:** $S_{i,j} = I_i \cdot T_j$

CLIP: Contrastive Language-Image Pre-training



Affective Vision Computing Lab

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter
```

```
# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]
```

```
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
```

```
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)
```

```
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

1. Image $\rightarrow$ Text InfoNCE Loss:

For the i - th image in the batch:

$$\mathcal{L}_{I \rightarrow T}^{(i)} = -\log \frac{\exp(S_{i,i}/\tau)}{\sum_{j=1}^N \exp(S_{i,j}/\tau)}$$

Averaging over all images in the batch yields:

$$\mathcal{L}_{I \rightarrow T} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{I \rightarrow T}^{(i)} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(I_i \cdot T_i/\tau)}{\sum_{j=1}^N \exp(I_i \cdot T_j/\tau)}$$

2. Text $\rightarrow$ Image InfoNCE Loss:

$$\mathcal{L}_{T \rightarrow I}^{(j)} = -\log \frac{\exp(S_{j,j}/\tau)}{\sum_{i=1}^N \exp(S_{i,j}/\tau)}$$

Averaging over all texts in the batch yields:

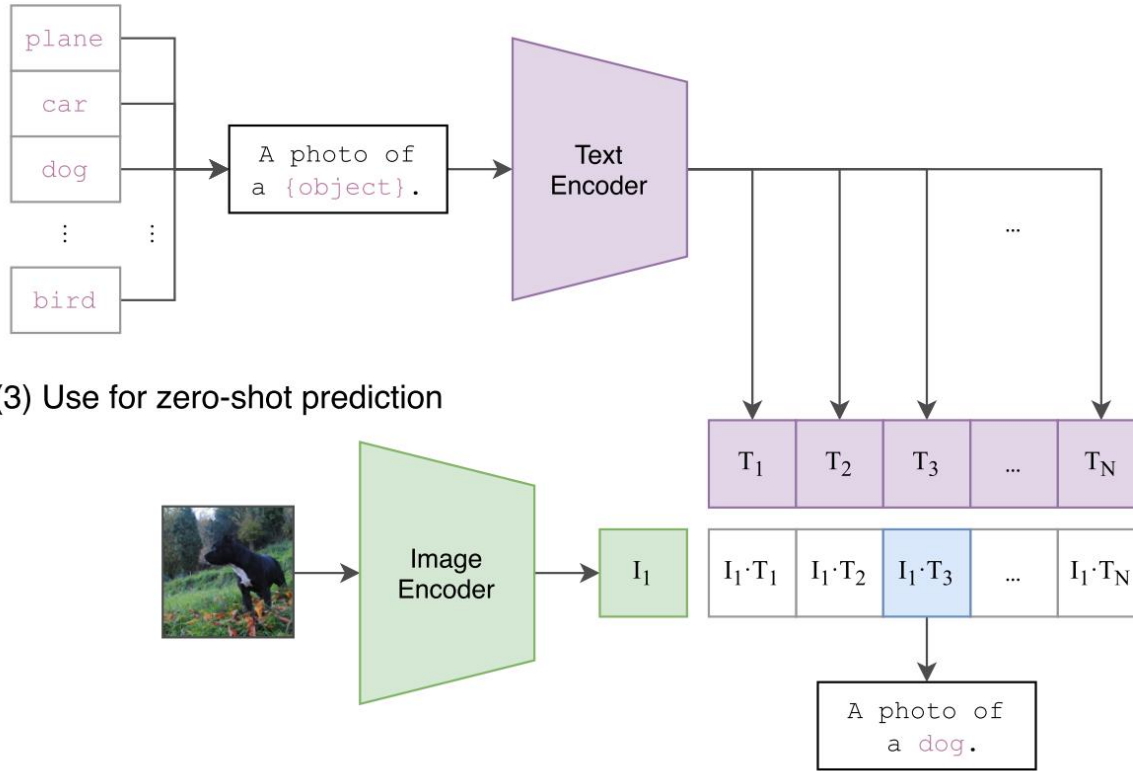
$$\mathcal{L}_{T \rightarrow I} = \frac{1}{N} \sum_{j=1}^N \mathcal{L}_{T \rightarrow I}^{(j)} = -\frac{1}{N} \sum_{j=1}^N \log \frac{\exp(I_j \cdot T_j/\tau)}{\sum_{i=1}^N \exp(I_i \cdot T_j/\tau)}$$

3. CLIP's Final Symmetric Joint Loss:

$$\mathcal{L}_{CLIP} = \frac{\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I}}{2}$$

CLIP: Contrastive Language-Image Pre-training

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

Test Time

Prompt Engineering:

Pre-training uses wild sentences instead of single words. Prompting "A photo of a {label}." aligns the evaluation data with the pre-training distribution, boosting ImageNet accuracy by 1.3%.



吉林大学
JILIN UNIVERSITY

THANKS